# 2212 Java Software Dev I
# Program Project 3 – Game Character Class

## Purpose

To practice the design, creation, and testing of classes.

## Goal

To develop a class that accurately represents characters in a computer game.

## Game Character Project Overview

Design, code, and test a class representing computer game characters. While it is perhaps useful to think about a specific character at first, your class design should be flexible so that we can use the class to create objects to represent all the different characters in a game. The attributes should be common to all the characters, while the actual value of those attributes will be different.

The characters must have:

- At least 5 attributes. Think of attributes all characters have, like a name, health, strength, etc.
- At least 1 constructor that receives parameters for the starting values of some/all attributes.
- a `toString` method that returns a String that describes the status of the character. (More about `toString` later in this document.)
- appropriate '`getter`' and '`setter`' methods for the attributes.
- An '`attack`' method and a '`defend`' method that use random numbers in some way.

## Process and Requirements

The first step is to create a class design showing the attributes, behaviors, and constructor for the game character class. The class design document must be submitted before you begin coding your solution. Things often change over time from the design phase to coding completion, so the final solution does not have to match the original design document exactly, but it should be close.

Then write the code for the class. The attack method must return a number that represents the strength of the attack. Use instance variables (for example, strength, current weapon, etc.) and a random number to calculate the attack strength value. The defend method must receive a parameter that represents the strength of the attack it is defending against. The defend method should use a calculation using instance variables and a random number to degrade health and/or other attributes.

*AI Tip*: Attempt a solution for the attack and defend methods first. Then do some research to see if there is a better way.

Finally, write some code in the main method to test your class. Test every method, either directly or indirectly. Create two objects with different attributes and have the two objects attack and defend for either a certain number of iterations or until one of the characters loses.

You may turn this into an actual game if you wish, but that is not what is required. Simple testing logic to prove that the class is written correctly will suffice.

## Explanation of the method `toString`

Later in the course, we will discuss class inheritance, but there is an aspect of inheritance we need to know for this project. There is a method that every object has named `toString`. The reason every object has such a method is that every class inherits from a class in the Java library named `Object`. Class `Object` has a few methods that all objects need, and `toString` is one of them. The version every object inherits is of limited use, so almost every class writes its own version of `toString` so that it is more useful in objects of that class. This is called 'overriding', which we will discuss in more detail later in the course.

To create your own version of the `toString` method, write the method like this:

```
public String toString() {
    return ???;
}
```

Where '???' is a `String` that you build with important information from the object. For example, if we write a class named `Student` that has a `String` variable named `myName`, our `toString` method for class `Student` might look like this:

```
Public String toString() {
     return "My name is " + myName;
}
```

The beauty of `toString` is that other methods know how and when to use it. For example, if we have a `Student` object named 's' and we have the following line of code:

```
System.out.println(s);
```

the `println` method knows to call the `toString` method from class Student, which will then output, "`My name is Sue`", or whatever value is stored in the variable `myName`.

After you write the `toString` method for your `GameCharacter` class, you should be able to send instances of your class to `println`, and it will display whatever String you built in the `toString` method.

*AI Tip:* Research various examples of how to implement the `toString` method.

## Expectations

The program is expected to compile successfully, include identification comments at the top, and produce output that exemplifies good grammar and spelling. Failure to meet these standards may result in point deductions.